

Input/Output and Interface Signal List

Low-level control (gripper hardware) ↔ High-level control (PC-side nodes)

This lists every signal exchanged between the low-level side (gripper hardware) and the high-level side (PC nodes) in the current simulation, plus how the same signals and functions would split across real hardware once this moves off MuJoCo: an MCU handling the gripper locally, the arm running on KUKA's Sunrise controller, and the PC keeping vision and supervisory control. All of it currently goes through one OPC UA server, and that stays true in the real deployment too.

LOW-LEVEL (gripper)		HIGH-LEVEL
Simulation: Sim Node Real deployment: MCU	↔ OPC UA Server	Simulation: Vision + Control Node Real deployment: PC (vision, supervisory logic) + KUKA Sunrise controller (arm motion)

1. Signals: Low-Level → High-Level (Outputs / Telemetry)

Signals originating at the gripper/hardware level (sensed or measured on the physical plant) and published upward for the PC-side nodes to consume.

Signal	Type	Publisher	Consumer(s)	Description
ArmActualQpos	Array[7] Real (rad)	Sim Node	Control Node	Actual joint positions read back from the physics simulation
UltrasonicDistance	Real (m)	Sim Node	Vision Node, Control Node	Live rangefinder distance from the wrist-mounted sensor
LeftForce	Real (N)	Sim Node	Vision Node	Left finger contact-force magnitude
RightForce	Real (N)	Sim Node	Vision Node	Right finger contact-force magnitude
AvgForce	Real (N)	Sim Node	Vision Node, Control Node	Average of Left/RightForce; used as the closed-loop grip-force feedback signal
CameraFrame	Bytes (JPEG)	Sim Node	Vision Node	Wrist-camera frame, republished every 16 simulation steps (~31 Hz)
GripperActualPos	Real	— (reserved)	—	Declared on the server but not currently written by any node

In deployment, ArmActualQpos comes from the Sunrise controller's own joint encoders rather than a Sim Node. UltrasonicDistance, LeftForce, RightForce and AvgForce come off the gripper's local sensors, read and pre-processed on the MCU before being sent up. CameraFrame keeps going through OPC UA as well, same as now — the camera is captured on the PC and published to the server so Vision Node picks it up the same way it does in simulation.

2. Signals: High-Level → Low-Level (Commands / Inputs)

Signals computed on the PC side and published downward as commands for the gripper/hardware level to execute.

Signal	Type	Publisher	Consumer(s)	Description
ArmTargetCtrl	Array[7] Real (rad)	Control Node	Sim Node	Commanded joint angles for all 7 arm joints
GripperTargetCtrl	Real	Control Node	Sim Node	Commanded gripper actuator position
SpeedFactor	Real (0.0-1.0)	Control Node	Vision Node	Safety-zone speed scale: stop / 15% / 40% / 100%, based on live proximity
CurrentPhase	String	Control Node	(none currently —available for HMI)	Human-readable state string, e.g. "FORCE_GRIP Zone: YELLOW (40%)"
ControlConnected	Bool	Control Node	(none currently —available for HMI)	Heartbeat flag, set True once the Control Node connects to the server

ArmTargetCtrl goes to the Sunrise controller instead of a Sim Node — the PC would send target poses or joint angles and Sunrise handles execution locally, since it has its own real-time loop. GripperTargetCtrl and the force setpoint go to the MCU. SpeedFactor, CurrentPhase and ControlConnected stay PC-side either way, since the safety-zone logic and sequencing don't need to run on either controller directly.

3. Signal Logging and Code Distribution in Real Deployment

Every signal moves between nodes live over OPC UA and, on its own, disappears once the process closes. A Logging Node (see the code alongside this PDF) fixes that on the PC side: it subscribes read-only to every signal below and appends a timestamped row to a CSV file, so there's a record of what the gripper did, when it gripped, how much force it read, and what the arm was doing at each point.

Logging split:

- **MCU side** — samples force and distance much faster than the PC polls them. Keeping every sample on the MCU isn't useful, so it keeps a short rolling buffer locally and only forwards the values needed for control (current reading, or a lightly smoothed one) up over OPC UA.
- **PC side** — this is where the actual log lives: **logging_node.py** polls every signal at 20 Hz, timestamps each row, and appends it to **signal_log.csv**. It runs as its own process, separate from Control Node, so logging never blocks or slows down the control loop, and it never writes back to the server — read-only, same as Vision Node.
- **Sunrise side** — the controller keeps its own internal logs (joint torques, safety events) which can be pulled separately if needed, but for this project the PC-side log covering ArmTargetCtrl / ArmActualQpos is enough to reconstruct what the arm was asked to do and what it actually did.

signal_log.csv columns: timestamp, ArmTargetCtrl, GripperTargetCtrl, ArmActualQpos, GripperActualPos, UltrasonicDistance (laser sensor), LeftForce, RightForce, AvgForce, SpeedFactor, CurrentPhase, ControlConnected. CameraFrame is left out of the CSV — raw JPEG bytes in a spreadsheet column aren't useful; the camera feed itself is already visible live through Vision Node's HUD.

Where the code actually runs:

Platform	Runs
MCU (gripper)	Force sensor + laser distance sensor reads, gripper motor/servo control loop (the closed-loop force-grip logic currently in Control Node moves here, since it needs a tight loop the PC round-trip over OPC UA can't reliably give it), local telemetry buffering.
KUKA Sunrise (arm)	Joint-level motion control and trajectory execution for the iiwa. The PC sends target joint angles or poses (equivalent of ArmTargetCtrl); Sunrise runs its own control loop and safety monitoring and reports actual joint state back.
PC (station computer)	Vision Node (camera capture + YOLO), Control Node (sequencing, safety-zone speed scaling, talking to Sunrise and the MCU), and Logging Node (writes signal_log.csv), plus the OPC UA server — all still on the PC, same as in simulation.

4. Node Roles, Functionality, and Interfaces

OPC UA Server

Level: Communication backbone (between levels)

Role: Holds every shared variable and passes reads/writes between whichever nodes need them, so the low and high side don't need to know how the other is built. Same role whether the low side is a Sim Node or an actual MCU.

Functionality:

- Registers all arm/gripper command, actual-state, telemetry, and status variables at startup.
- Makes every variable writable so any connected client can read or update it.
- Runs standalone; must be started before any other node connects.

Publishes	initial values for all registered variables
Subscribes to	—

Sim Node — Low-Level Control (gripper/arm hardware)

Level: Low-level (gripper/arm hardware) — stands in for the MCU during simulation

Role: Plays the role of the physical plant: applies commanded actuator targets in MuJoCo and reports back sensor readings. In the real system this is what the MCU does instead, reading the actual force and distance sensors and driving the gripper motor.

Functionality:

- Applies ArmTargetCtrl / GripperTargetCtrl directly as actuator control targets each simulation step.
- Steps the MuJoCo physics simulation.
- Reads back finger force sensors and the rangefinder each step and publishes them.
- Renders the wrist camera and publishes the encoded frame every 16 steps — in deployment this is still published over OPC UA the same way.

Publishes	ArmActualQpos, UltrasonicDistance, LeftForce, RightForce, AvgForce, CameraFrame
Subscribes to	ArmTargetCtrl, GripperTargetCtrl

Vision Node — High-Level Control (PC side)

Level: High-level (PC side)

Role: Reads the camera feed and live telemetry, runs YOLO detection, and shows a live HUD for monitoring. Doesn't publish anything back to the server right now — it's read-only.

Functionality:

- Subscribes to the camera frame and runs YOLO inference on each frame.
- Ray-casts detected bounding-box centers to an approximate 3D world position on the table plane.
- Overlays a live HUD (distance, per-finger and average force, current safety zone) on the video feed.

Publishes	—
Subscribes to	CameraFrame, UltrasonicDistance, LeftForce, RightForce, AvgForce, SpeedFactor

Control Node — High-Level Control (PC side)

Level: High-level (PC side)

Role: Runs the pick-and-place sequence between fixed waypoints (frame-to-frame, not IK-driven), a closed-loop force controller for gripping, and the safety-zone speed limiter. In deployment, the force-grip loop moves to the MCU and the waypoint execution goes through Sunrise instead of straight into a physics step.

Functionality:

- Interpolates joint angles between waypoints using a smoothstep ease, scaled by the live safety speed factor.
- Computes a stop / 15% / 40% / 100% speed factor from live laser distance and commanded velocity (SSM-style safety logic).
- Runs a closed-loop force controller that closes the gripper until AvgForce reaches a target force, then holds.
- Publishes a human-readable phase string and a connection heartbeat for external monitoring.

Publishes	ArmTargetCtrl, GripperTargetCtrl, SpeedFactor, CurrentPhase, ControlConnected
Subscribes to	UltrasonicDistance, AvgForce, ArmActualQpos

Logging Node — High-Level Control (PC side)

Level: High-level (PC side)

Role: Read-only, same as Vision Node: subscribes to every telemetry and status signal and appends a timestamped row to signal_log.csv, at 20 Hz. Runs as its own process so it never adds latency to the control loop.

Functionality:

- Polls all eleven telemetry/command/status signals every 50 ms.
- Writes one CSV row per poll: timestamp plus each signal's current value.
- Creates signal_log.csv with a header row on first run; appends on every run after that.

Publishes	—
Subscribes to	ArmTargetCtrl, GripperTargetCtrl, ArmActualQpos, GripperActualPos, UltrasonicDistance, LeftForce, RightForce, AvgForce, SpeedFactor, CurrentPhase, ControlConnected